

GOSIM Paris 2026

May 5-6, 2026 Station F, Paris



openJiuwen: JiuwenClaw
pioneers “Coordination Engineering” , the
next leap for Agent Era



CONTENTS

GOSIM Paris 2026

Part 01

openJiuwen Introduction

Part 02

JiuwenClaw Introduction

Part 03

Coordination Engineering



01 openJiuwen Introduction



Making Production-Ready AI Agents Development Easier — openJiuwen Agent Platform, become Open Source now!

GOSIM Paris 2026

ToB

Industry-Specific Agent



Finance

Intelligent Customer Service...



Healthcare

Disease Diagnosis...



Insurance

Intelligent Insurance Advisory ...



Electric Power

Anomaly Detection...

...



Huawei Cloud
AgentArts



Celia Open
Platform

APP Agent



Celia Premium Agents

Task Space / Travel Assistant ...



Application Intelligence Upgrade

Gallery / Music / Browser...



Ecosystem Agents

Lifestyle / Healthcare / Mobility...

...

ToC



• openJiuwen Official Website : <https://www.openjiuwen.com> • openJiuwen Project Repository : <https://github.com/openJiuwen-ai>

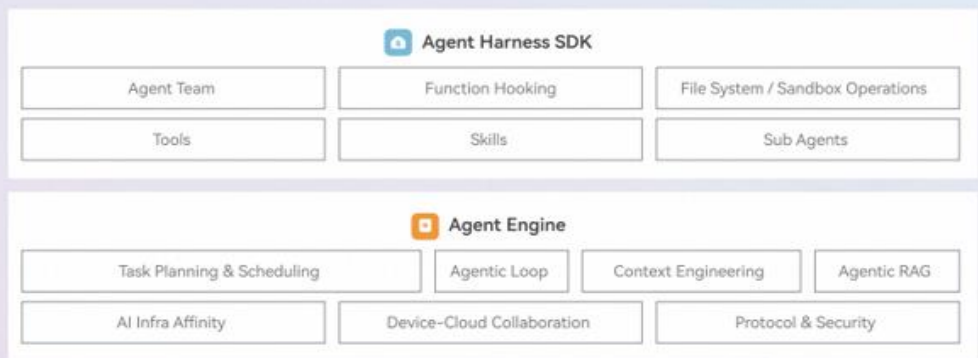
The openJiuwen AI agent platform provides a one-stop solution for agent development, deployment, operation, monitoring, and optimization. It is dedicated to making agent development easier, operation more efficient, processes more controllable, and outcomes more precise.

The openJiuwen features an innovative self-evolving technical architecture, precise multi-workflow control, a highly reliable execution foundation, and automated prompt generation and optimization. openJiuwen meets the full-stack needs of everyone—from beginners in AI agents to seasoned Agentic AI developers.

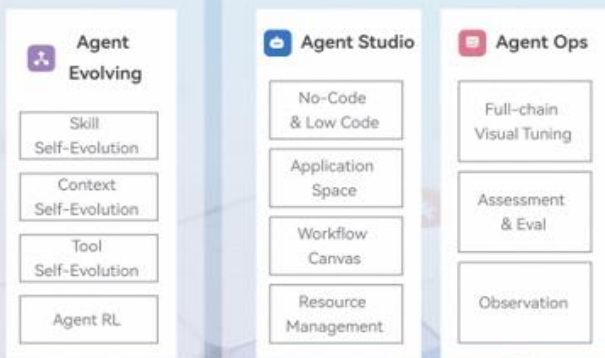
DeepAgents



Agent-Core



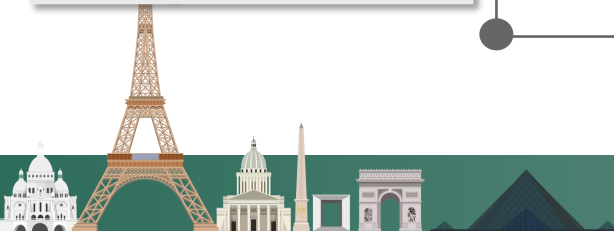
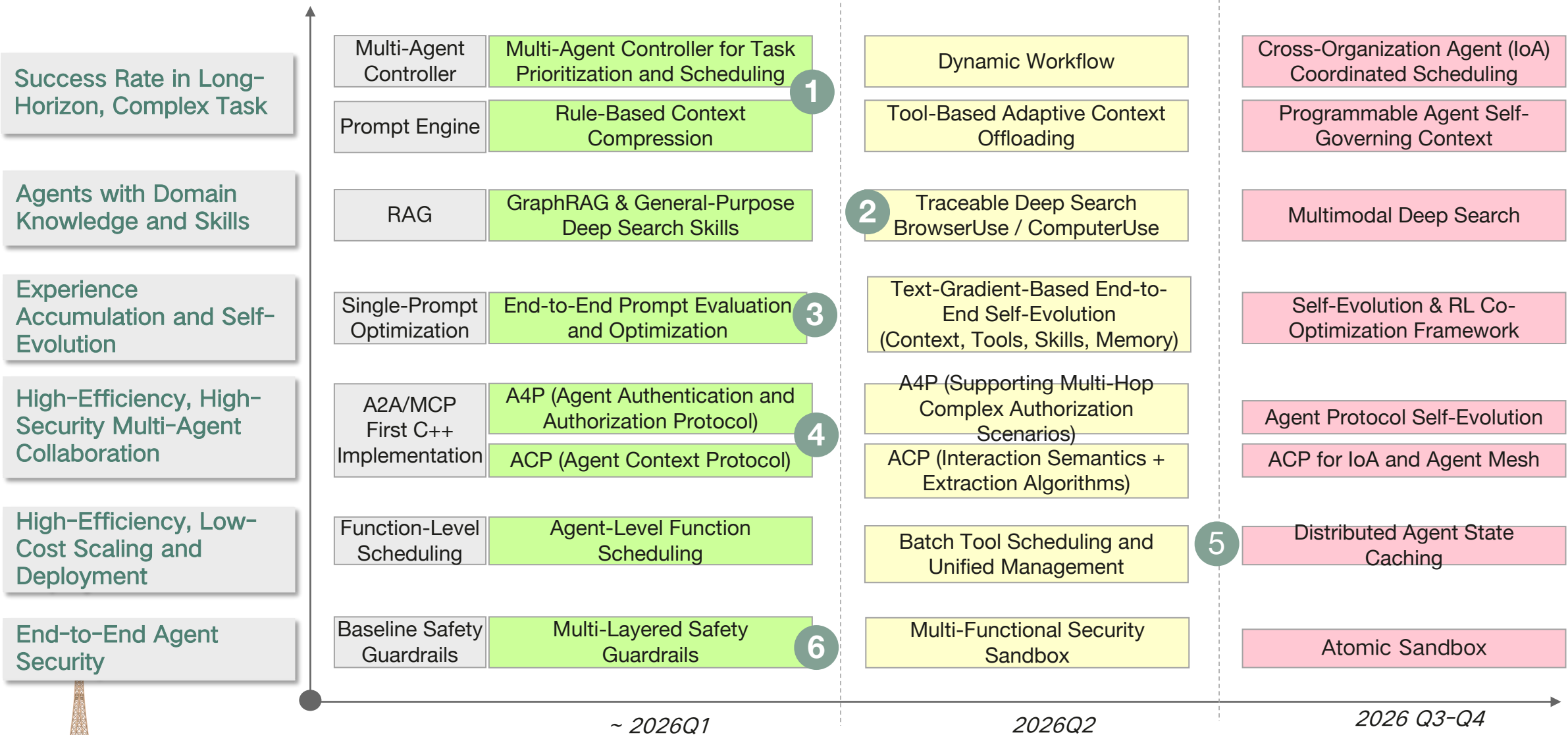
Agent-Studio & Agent-Ops

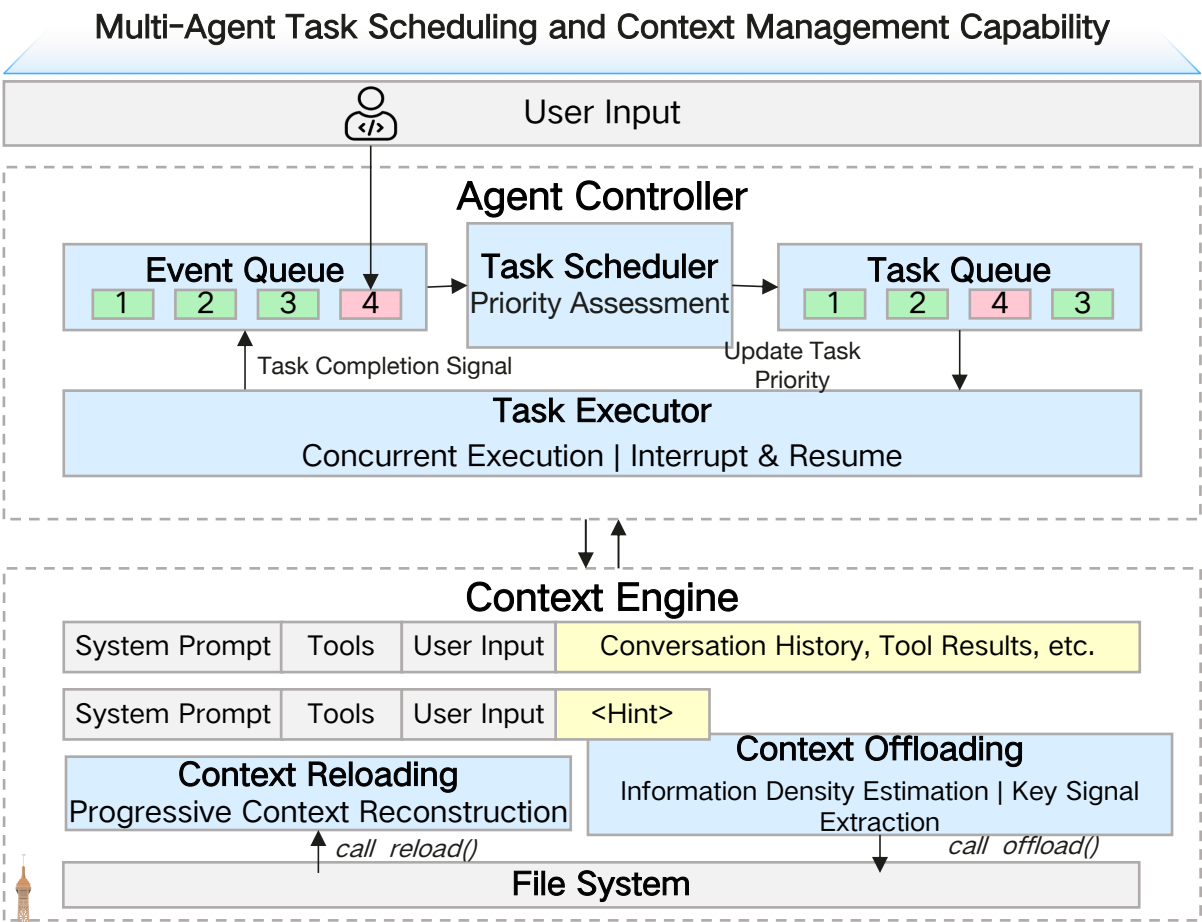


Agent-Service



Six Key Technology Directions for Next-Generation Agents Aligned with Industry Trends



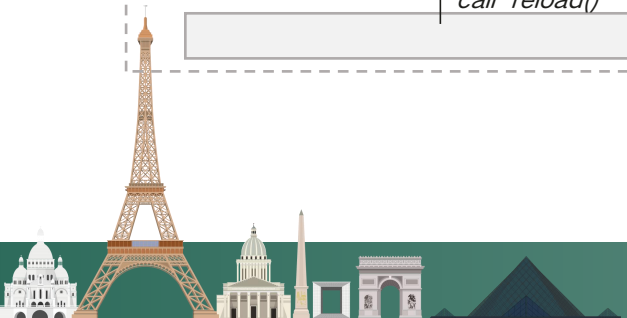


Current Challenges

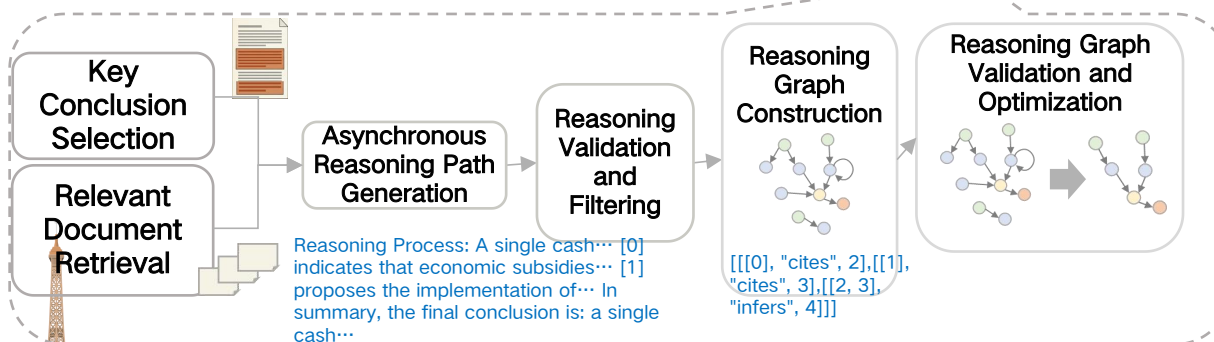
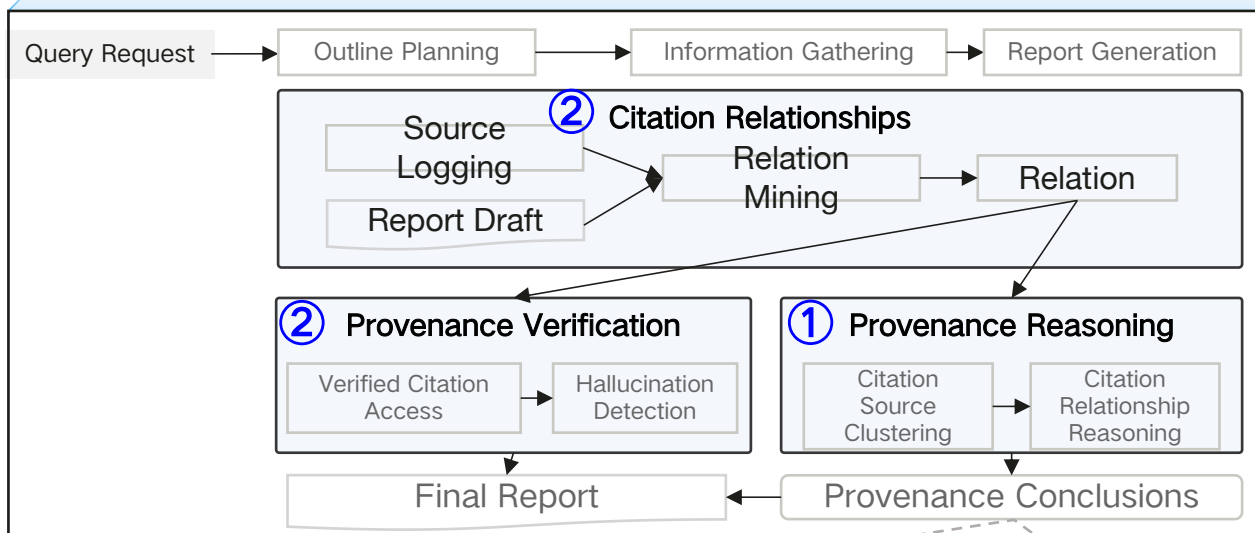
- Most frameworks (e.g., OpenClaw) use synchronous queues without priority-aware, preemptive multi-agent scheduling, resulting in high failure rates for complex tasks.
- Context management relies on summarization or truncation, making lost details unrecoverable and causing downstream tasks to miss critical information.

Key Technologies & Core Competencies

- ① **Multi-Agent Controller for Task Prioritization and Scheduling**
 - Supports priority-preemptive scheduling to improve complex task success rates and resource utilization efficiency
 - Asynchronous, event-driven parallel scheduling to eliminate serial bottlenecks and improve execution efficiency for complex tasks
- ② **Information-Density – Based Adaptive Context Offloading**
 - Information Density Estimation: Preserve high-density semantic signals during early offloading based on information entropy theory
 - On-Demand Reloading: Use semantic cues as index keys to reconstruct context as needed, ensuring downstream semantic integrity



Traceable DeepSearch Capability

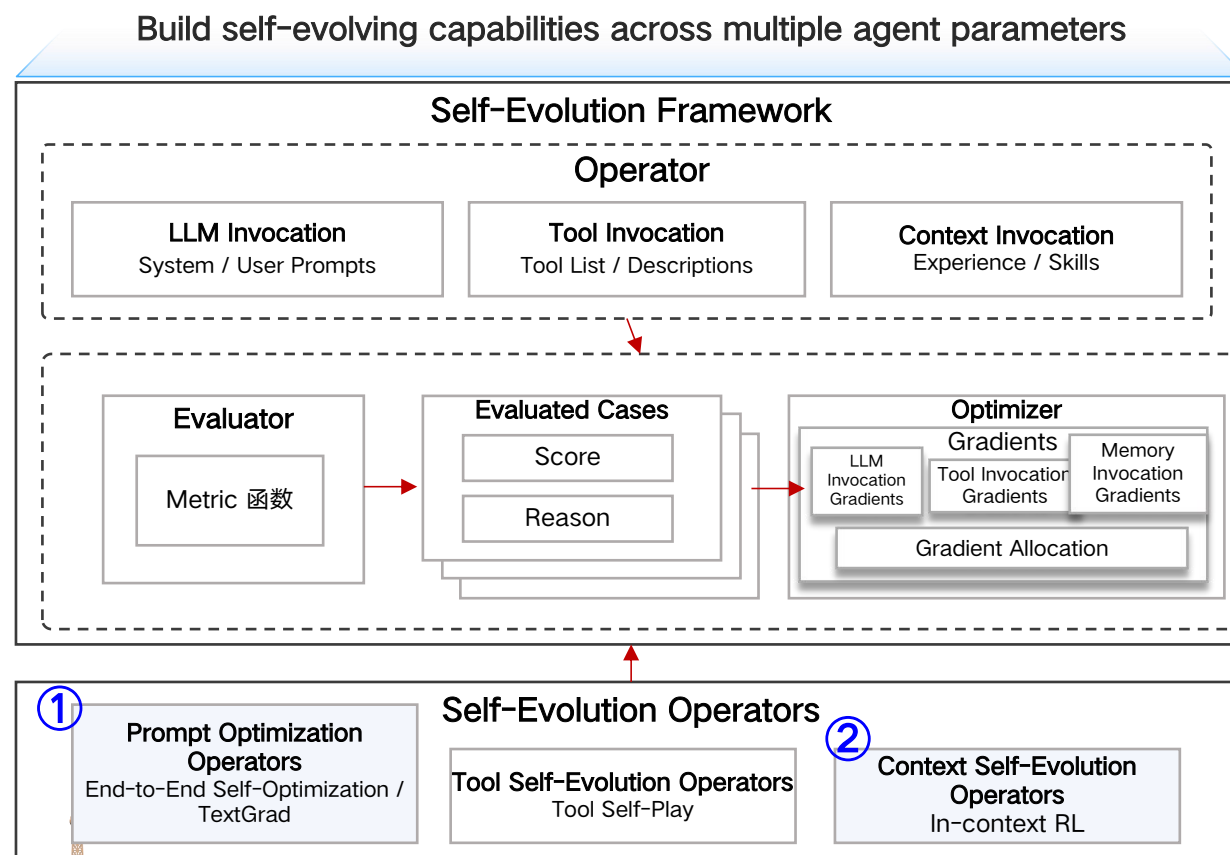


Current Challenges

- **Broken provenance chains after multi-turn compression:** repeated compression and rewriting in deep search make final conclusions untraceable to sources, rendering them unauditible and unsuitable for regulated industries.
- **Coarse-Grained Attribution:** Mainstream solutions (e.g., Perplexity AI, Google, OpenAI) provide only URL-level citations; with 50-page reports or long webpages, users must still locate the relevant passages themselves.

Key Technologies & Core Competencies

- ① **Knowledge-Graph-Based Structured Provenance Reasoning:** Abstract key conclusions and supporting documents into triples of “<conclusion, semantic relation, evidence segment>,” constructing a provenance knowledge graph where reasoning paths are explicitly represented as graph structures, enabling auditable inference chains
- ② **Fine-Grained Segment-Level Attribution:** Pre-segment documents into addressable semantic units with unique IDs, establish structured “conclusion – segment ID” mappings during generation, and enable hallucination detection and correction via provenance verification

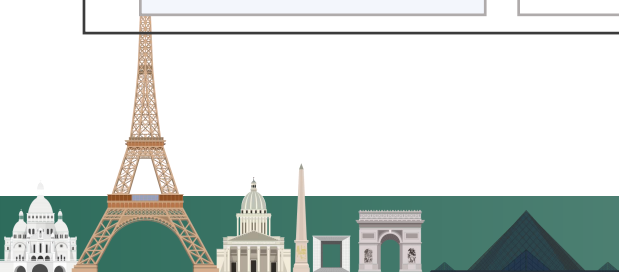


Current Challenges

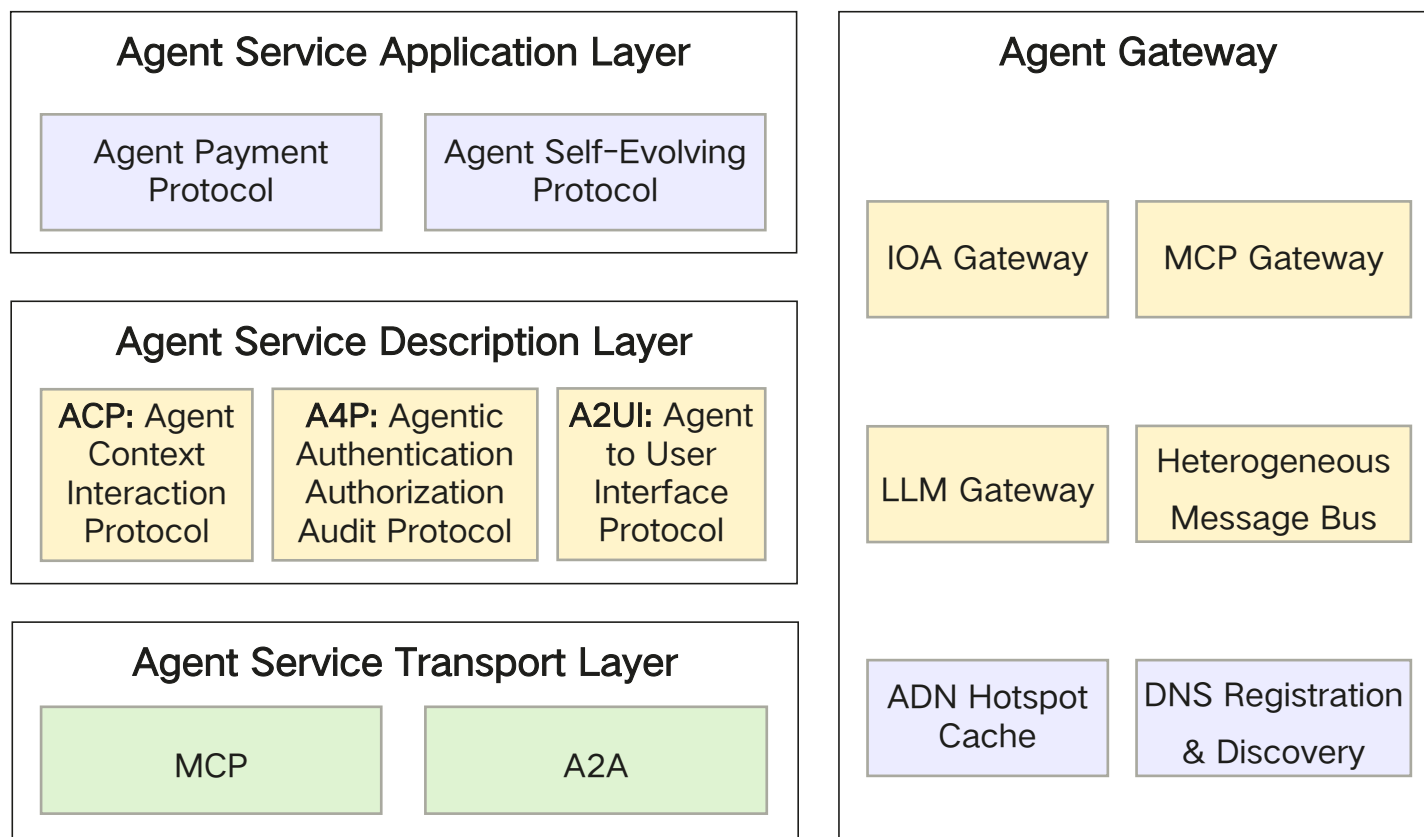
- **Lack of global optimization:** Current agent tuning remains pointwise; success depends on coupled factors (context, tool use, etc.), so single-point optimization cannot achieve global optimum
- **Continuous optimization with minimal human intervention:** Enable agents to improve overall task performance over time while minimizing manual involvement

Key Technologies & Core Competencies

- ① **End-to-End Prompt Self-Optimization:** Unify prompts across multi-step agent trajectories into a single optimization space, enabling joint multi-node optimization via text-gradient backpropagation
- ② **In-context RL:** Continuous behavior improvement without updating model parameters by injecting historical trajectories, reward signals, or reflection into the context



openJiuwen A2X protocol



Deployed

In Progress

In Planning

Key Technologies & Core Competencies

① MCP C++ SDK

- The industry's first C++ development framework with full-feature support for MCP, enabling highly efficient adaptation to tool service deployments and operation requirements across diverse scenarios.

② A2A C++ SDK

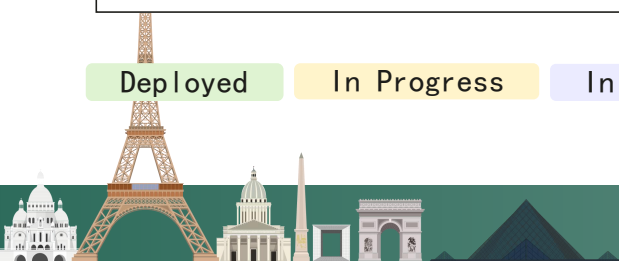
- Empowering Agent-to-Agent communication with C++ cross-platform collaboration capabilities, enabling seamless deployment of agentic collaboration services across diverse execution environments.

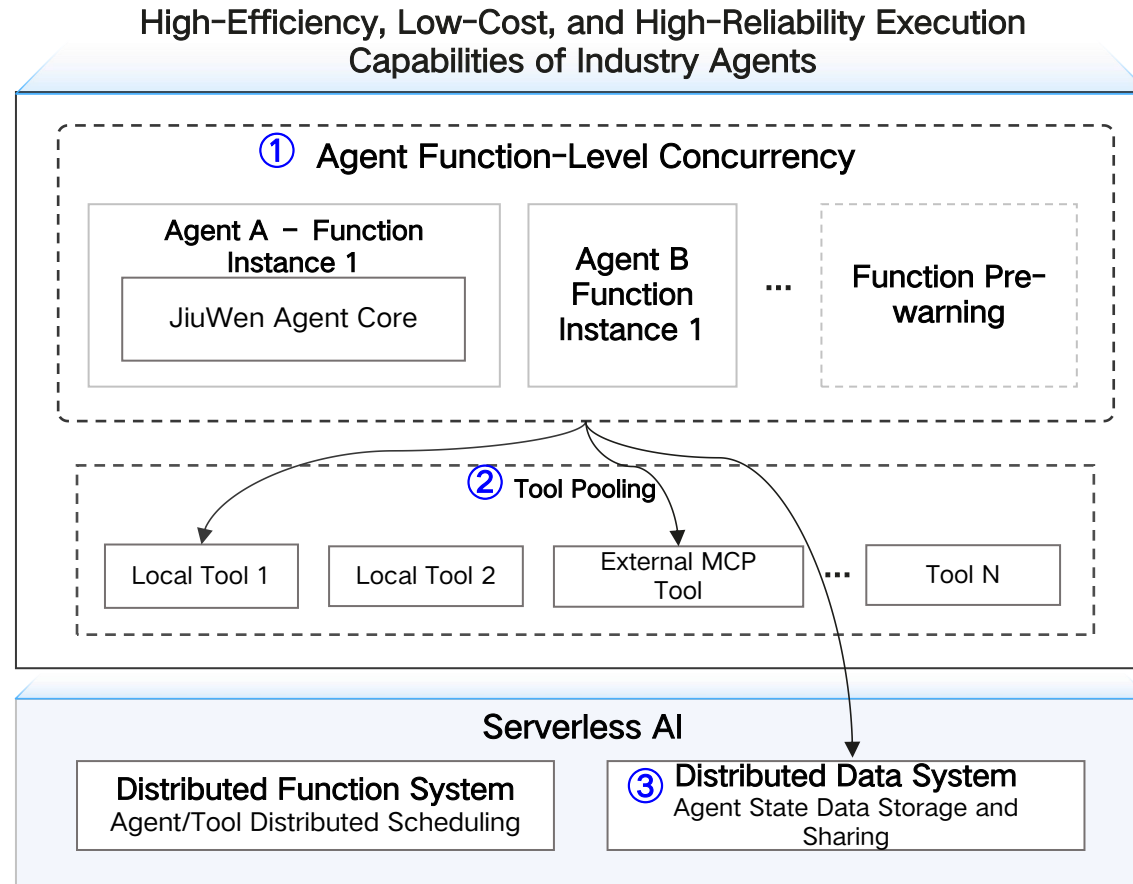
③ MCP Dynamic Authorization

- Featuring a versatile server-side framework for authentication and dynamic authorization, the system supports custom access control policies to meet the secure integration requirements for agents connecting to high-privileged MCP services.

④ OpenAPI to MCP

- Enabling unified integration of OpenAPI tools into MCP. This ecosystem-compatible and model-native approach significantly enhances developer usability.





Current Issues

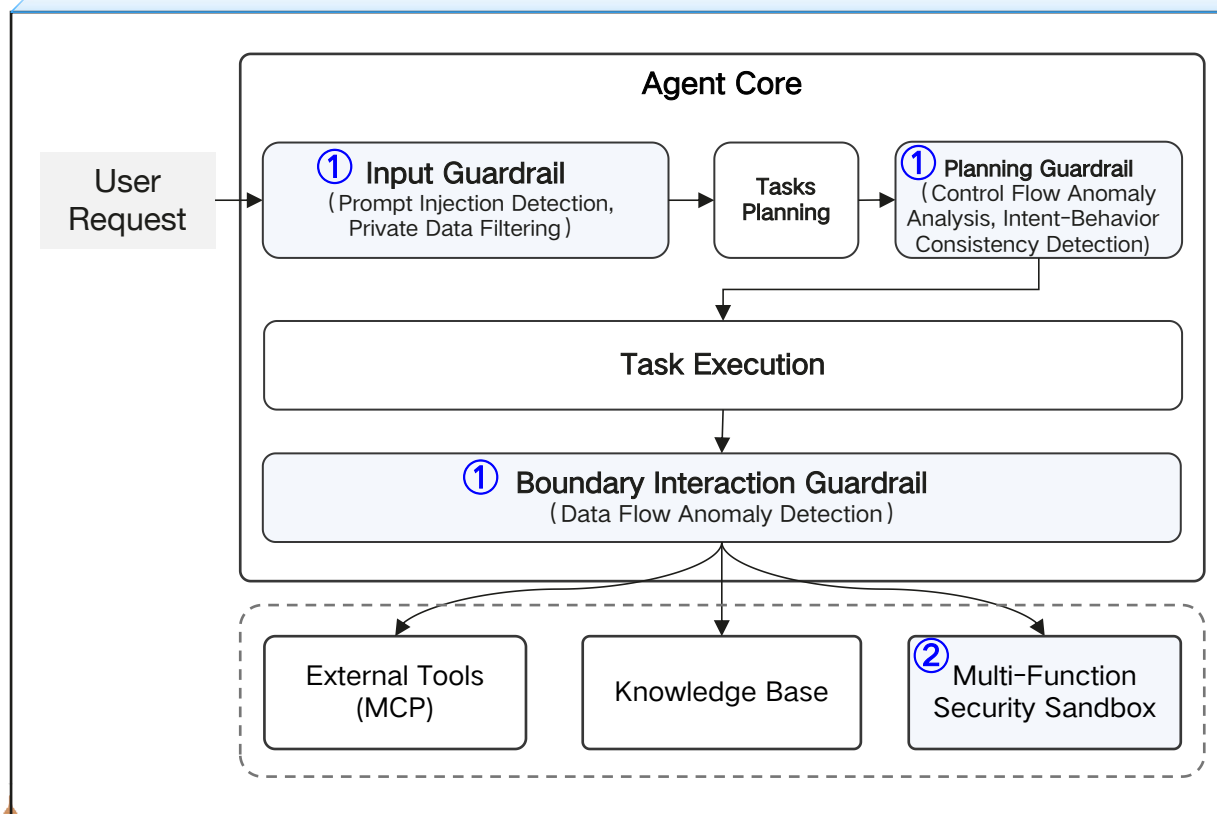
In complex scenarios, multi-step long-chain execution, high-frequency tool calls, multi-round interactions, and frequent state switching by agents can easily lead to issues such as insufficient Agent execution throughput, fragmented resource scheduling, low resource utilization, high latency fluctuations, and elevated infrastructure costs.

Key Technologies & Competitive Advantages

- ① **Agent Function-Level Concurrency:** Based on Serverless architecture to achieve Agent function-level scheduling and concurrent execution, supporting rapid response and elastic scaling, with throughput improvement of 50%
- ② **Tool Pooling:** Based on resource pooling technology to achieve batch scheduling and unified management of tools, reducing resource overhead by 30%
- ③ **Distributed Agent State Caching:** Based on distributed data systems to achieve persistent storage and efficient read/write of Agent execution states, ensuring high Agent availability (>99.9%).



Security Protection and Isolation Capabilities for Industry Agents

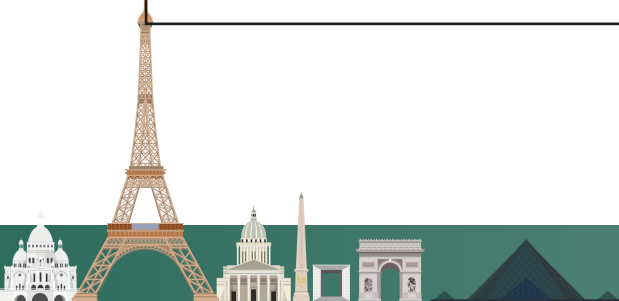


Scenarios & Challenges

- Agents engage in high-frequency interactions with users, large models, and tools across multiple parties, resulting in a large security exposure surface that is vulnerable to attacks such as malicious instructions and prompt injection.
- Execution-Level Risks: Agents involve high-risk behaviors such as code generation and execution, system operations, etc., presenting security vulnerabilities including malicious code execution and unauthorized privilege escalation.

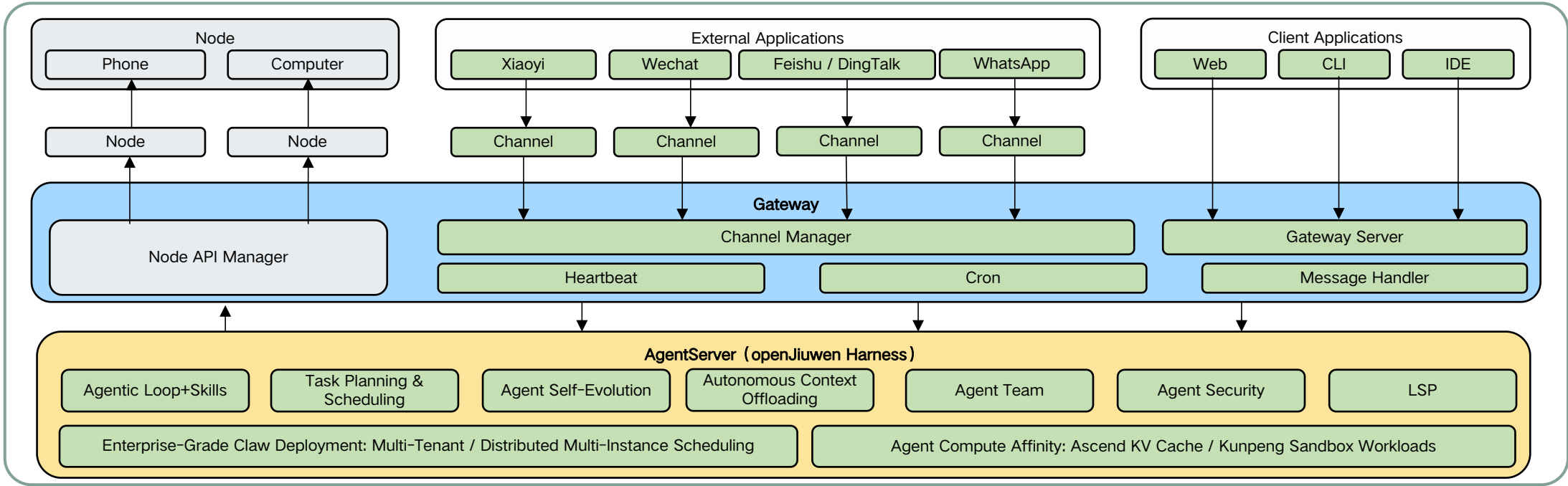
Key Technologies & Competitive Advantages

- ① Multi-Level Safety Guardrails:** Covering end-to-end security detection and protection across user input, task planning, and external interactions; enabling real-time detection and interception of high-risk behaviors during complex Agent task execution, with full-chain detection latency < 150ms
- ② Multi-Function Security Sandbox:** Providing integrated capabilities including code execution, file system operations, and GUI operations, meeting process-level and system-level security isolation requirements across multiple scenarios; leveraging sandbox pooling technology to achieve response latency < 100ms



02 JiuwenClaw Introduction





Core Framework

Built on the openJiuwen Python framework, it naturally offers strong ecosystem compatibility and extensibility. It is compatible with personal assistant and coding scenarios, supports self-hosted deployment, and integrates with mainstream Skills ecosystems.



Application Scenarios

- 1.Deep Local System Control & Automation:** System management, in-depth research, life services, and content creation.
- 2.Multi-Platform & Multimodal Interaction:** Supports platforms such as Xiaoyi, WeChat, Feishu, DingTalk, and Telegram.
- 3.Software Development:** Supports local deployment, coding, testing, refactoring, and cross-language conversion.



Core Features

Core modules operate in coordination, including task planning & multi-agent collaboration, self-evolution engine, context & memory engine, Ascend compute affinity, security mechanisms, and enterprise-grade deployment.



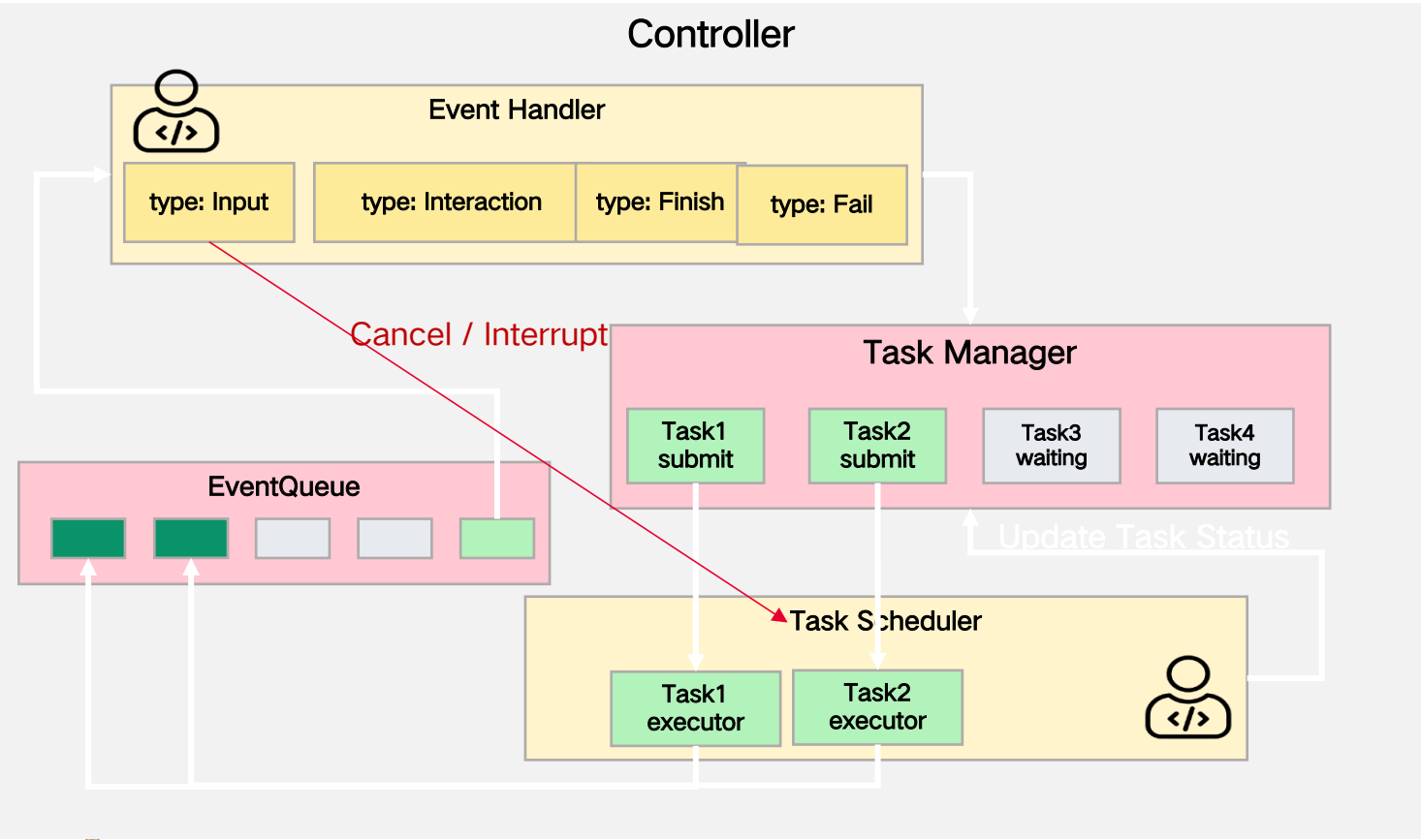
Ecosystem Usage

Deployed across major enterprise clients such as Postal Savings Bank, China Mobile, China Telecom, and the Shenzhen Municipal Bureau, as well as Huawei Cloud products including OfficeClaw and PC Claw. Integrated computing appliance solutions are currently under negotiation.



☰ Task Planning Mode

Provides agents with structured task decomposition and dynamic management through a systematic to-do framework, enabling orderly execution and clear objectives for complex tasks.



⏸ Dynamic Interruption

During task execution, the current process can be interrupted at any time to respond to new high-priority requests.

⏴ Priority Management

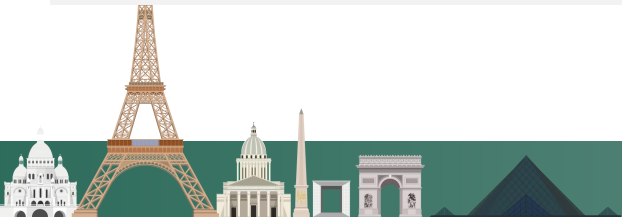
Intelligently prioritizes tasks to handle multiple requests in an orderly and efficient manner.

⊕ Task Injection

Allows new tasks to be inserted during execution without requiring re-queuing or reprocessing.

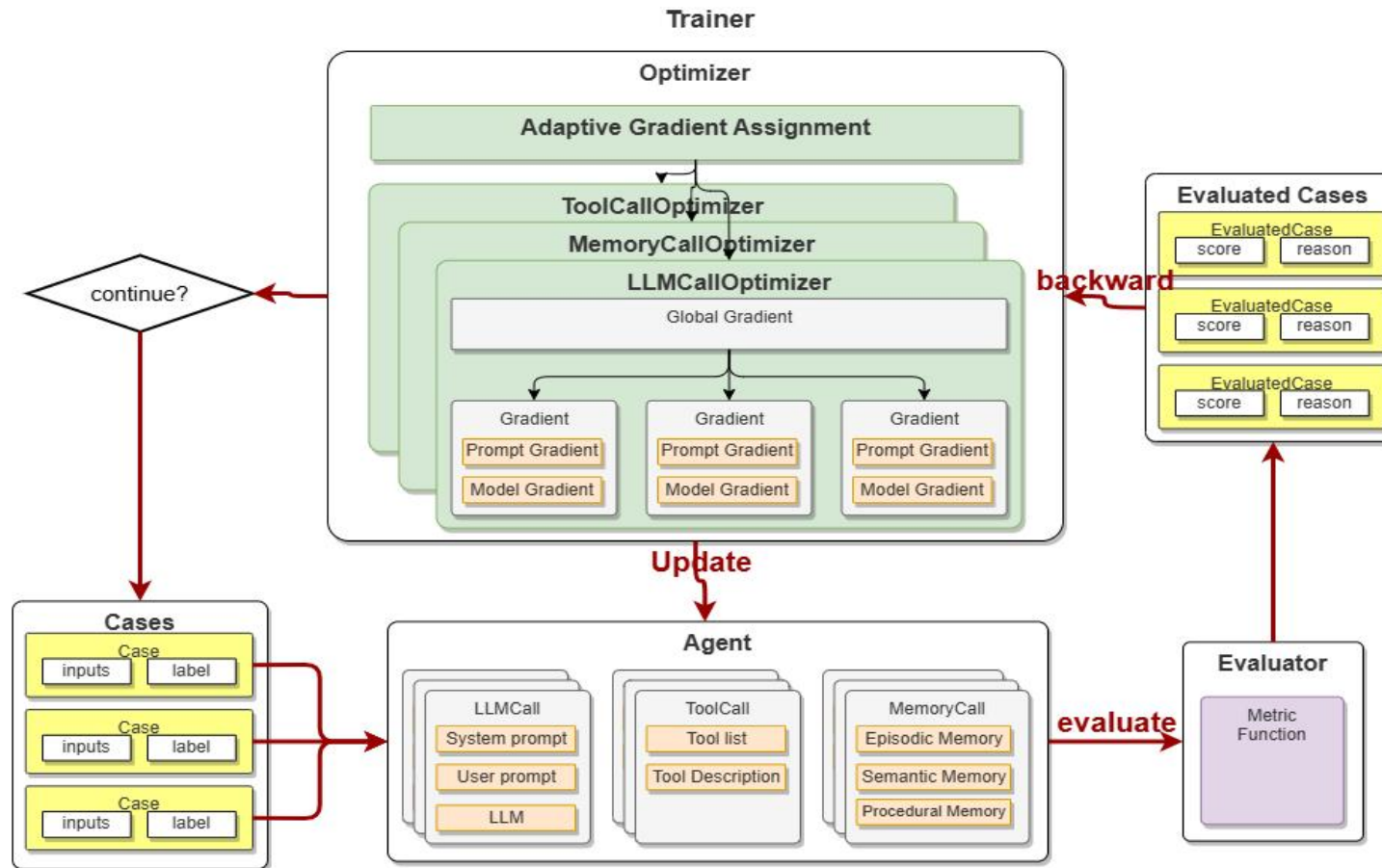
✎ Task Modification

Supports real-time adjustment and parameter updates for planned tasks.



Agent Self-Evolution

Built on the openJiuwen self-evolution framework, it provides unified management of Skill read/write operations and evolution dispatch, with a built-in evolution signal detection mechanism that continuously monitors execution processes and dialogue content.



⚠ Execution Anomalies

- > Tool Invocation Timeout
- > API Error Response
- > Code Execution Interrupted by Exception

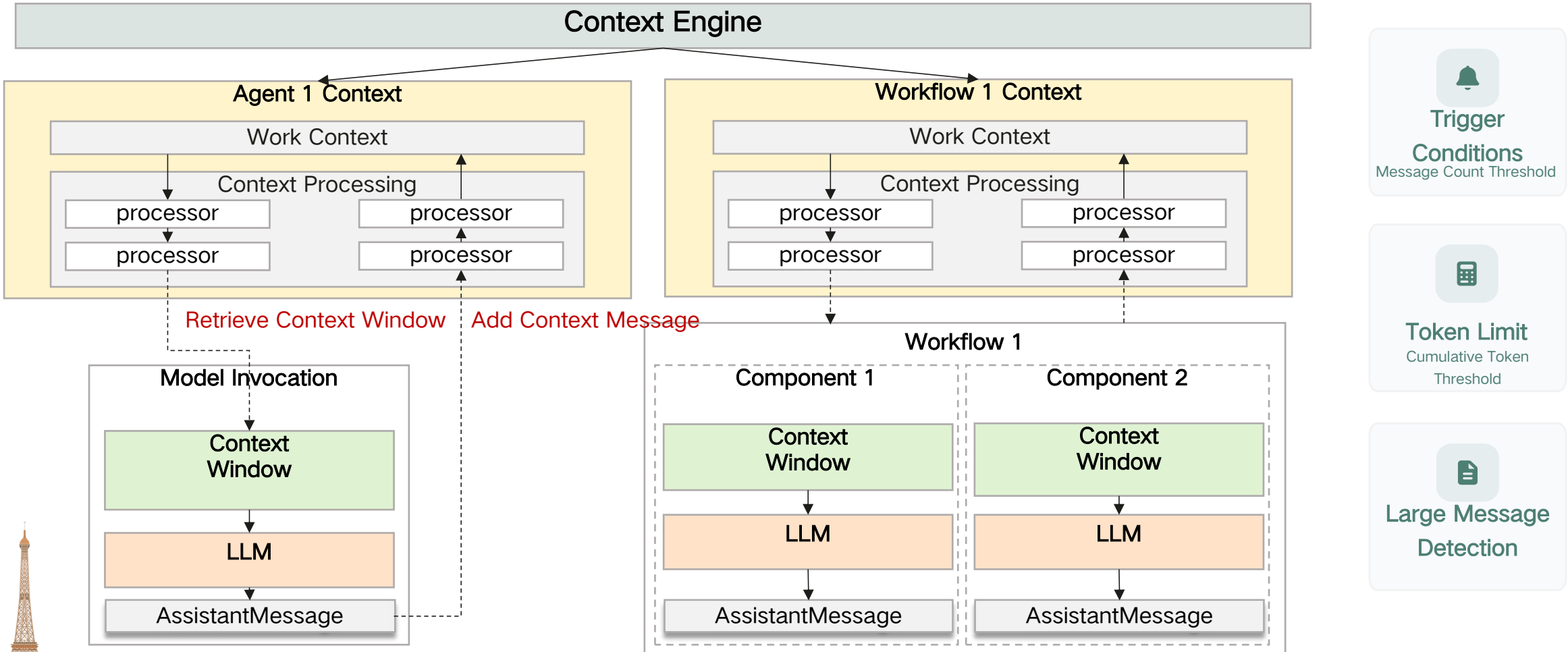
👤 User Correction

- > " That' s not right "
- > " We should try a different approach "
- > "You misunderstood"



✦ Dynamic Context Management

Core idea is like “tidying a desk” : when messages or tokens exceed a threshold, less important content is automatically archived and compressed, keeping only key indexes to free space for current tasks.



03 Coordination Engineering



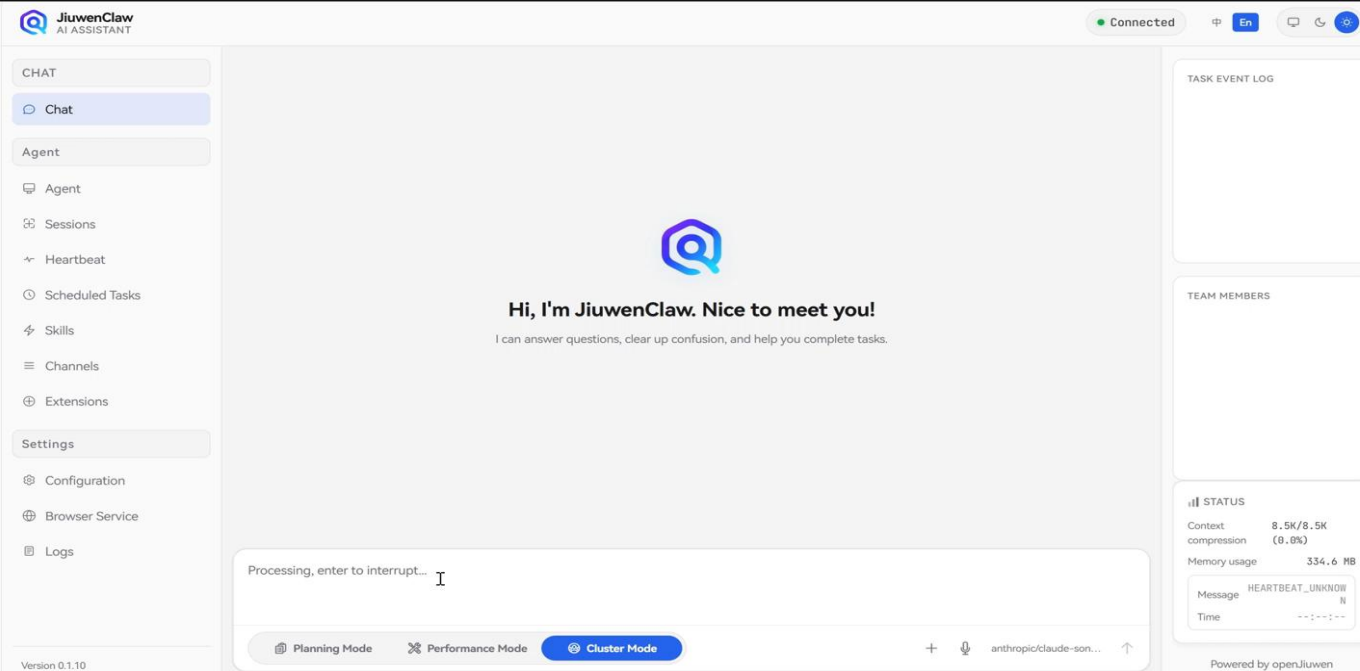
Coordination Engineering: The Industry-First Leap Beyond Harness Engineering

GOSIM Paris 2026

✦ From Harness → Coordination

- **Harness Engineering:** builds the infrastructure around a single agent (tools, memory, execution)
- **Coordination Engineering:** organizes multiple agents as a unified intelligent system
- **Goal:** Make agents collaborate like an elite autonomous team

Capability Showcase: Automatically generates 200-page technical PPT in < 20 minutes



Coordinate an Agent Team to complete complex tasks.

Core System Design

1. Hierarchical Orchestration: Leader agent builds and manages task graph dynamically
2. Shared Workspace & Memory: Unified context across agents for long-horizon tasks
3. Event-Driven Reliability: Async execution, fault recovery, robust pipelines

Self-Evolving Intelligence

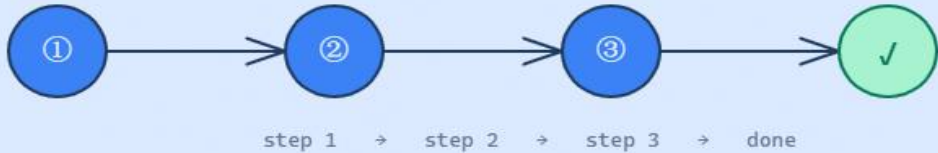
What JiuwenClaw Enables

1. Continuous feedback loops: Plan → Act → Evaluate → Improve
2. System improves team structure, strategies, and outputs over time
3. Moves from static pipelines
→ adaptive agent ecosystems

GOSIM

Agent skills

Solves: How a single Agent completes a single task (a linear list of steps)

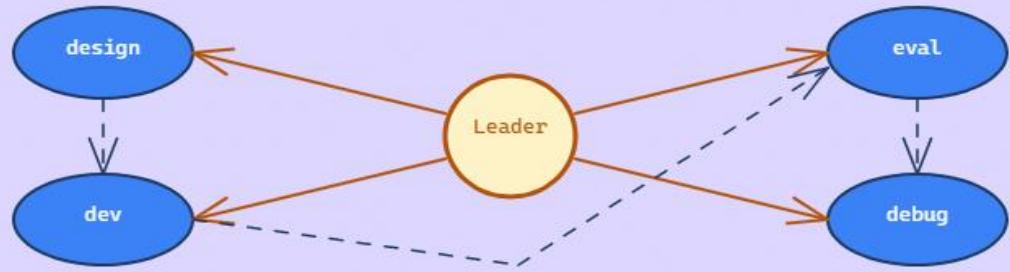


Just 1 SKILL.md

(optionally with references/ sub-docs, loaded via progressive disclosure)

Team Skills

Solves: How a team collaborates to complete a category of tasks (a network of roles + collaboration workflow + constraints + dependencies)



Packing: SKILL.md + roles/ + workflow.md + bind.md + dependencies.yaml ← the five-piece set

Full example: PR review team (3 roles in parallel · strictly follows the Team Skills Spec)

File structure – the five-piece set (cp -r to distribute, not tied to any framework)

```
team_skills/pr-review-team/
├── SKILL.md ← Entry point: trigger detection + overall workflow routing
│   └── The first file an Agent reads
├── roles/ ← One .md per role (filename = role id)
│   ├── code-reviewer.md
│   ├── critique-adversarial.md
│   └── architect.md
├── workflow.md ← Collaboration workflow: Mermaid topology + step-by-step + quality gates
│   └── Leader gets context → 3 roles review in parallel → produce ranked report
├── bind.md ← Execution guardrails: resource caps / behavioral baselines / failure fallbacks
│   └── ≤3 concurrent · 10 min · 200k tokens · no visibility into each other
└── dependencies.yaml ← External dependency pool (skills + tools), referenced by roles[].skills
    └── Verified before launch; missing items reported to the user
```

SKILL.md frontmatter – only two fields extended: kind + roles[] (id / purpose required · skills / tools optional)

```
name: pr-review-team
description: |
  Multi-role PR review team:
  optimistic reviewer + adversarial critic + architect.
  Use when: non-trivial PR (>200 LOC, >2 modules,
  security-sensitive).
  DO NOT use for: typo-only / doc-only / single-file refactor.
version: "0.1"
kind: team-skill           # ← key discriminator (Team Skill, not a single Skill)
roles:                     # ← which roles make up the team
  - id: code-reviewer
    purpose: Optimistic forward review – quality, readability, tests.
    skills: [code-search]    # ← references an external skill from dependencies.yaml
    tools: [gh, grep]
  - id: critique-adversarial
    purpose: Adversarial critic – risks, edge cases, future pain.
    tools: [gh]              # ← skills are optional; omit if not needed
  - id: architect
    purpose: System-level – boundaries, contracts, cross-cutting concerns.
    skills: [dependency-graph]
```


Team Skill lifecycle: from creation to codification

① Creation: trajectory → LLM distillation → approval → commit to disk



propose_team_skill prompt embeds three hard constraints

- ① De-concretize – domain-specific nouns (products, stacks, people) in the trajectory are extracted into parameters
Example: "Sinkhorn operator" → "{ALGORITHM}"
- ② Keep the skeleton – preserve role configuration, information flow (fan-out/in), trigger conditions, and acceptance criteria
- ③ Cross-domain self-check – apply this skill to two different domains for the same kind of task; does it still hold?
If no → too narrow, re-distill.
If it degenerates into generic boilerplate → too broad, add constraints.

■ Key argument: approval gate + three hard constraints → what gets codified is not a snapshot of a single run, but a reusable SOP for a category of tasks

② OPTIMIZATION: two-layer closed loop, reusing 1D Skill's EvolutionStore + ExperienceScorer verbatim

L1 – Automatic learning loop (high frequency) · runs after every task completion



Before the next task, the Leader loads SKILL.md and concatenates records in order of score (descending)

3-D scoring (reuses ExperienceScorer.calc_score verbatim)

$$\text{score} = E \times 0.5 + U \times 0.3 + F \times 0.2$$

E – Effectiveness (Beta(1,1) smoothing)
U – Usage rate (times_used / times_presented)
F – Freshness (90-day half-life)

L2 – Manual governance (low frequency) · shared by 1D / Team Skills, all changes go through approval

/evolve_simplify <skill>
[intent]
– Pruning –
DELETE / MERGE
REFINE / KEEP
Clean up low-scoring / similar patches

/evolve_rebuild <skill>
[intent]
– Regrowth –
Rewrite the SKILL.md body
Old body archived under archive/
Unlocks first-run lock

/evolve_rollback <skill>
[version]
– Fallback –
Rollback to any archived version
(No approval required; archive protection acts as the safety net)

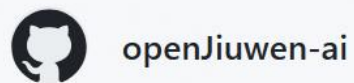
Three-tool positioning:

- 1. simplify → solves "decay" (low-scoring / duplicate / outdated patches piling up)
- 2. rebuild → solves "first-run lock" (the initial structure is wrong and subsequent patches can't change the skeleton)
- 3. rollback → safety net for rebuild/simplify mistakes (no approval required; archived versions can be restored anytime)

■ Two-layer closed loop = retains the speed of high-frequency automatic learning + two-layer approval / governance command family to ensure "no decay, rewritable, rollbackable" – the fundamental difference from Hermès's "mindless appending".



Code Repository



<https://github.com/openJiuwen-ai>

Official Website



<https://www.openjiuwen.com/>



Code Repository



<https://gitcode.com/openJiuwen>



Official Media

openJiuwen Official Account



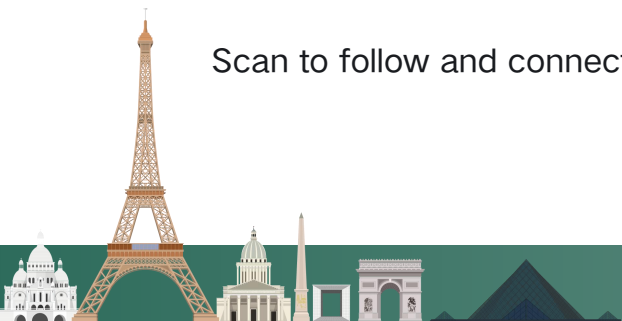
openJiuwen Video Channel



openJiuwen Developer Community



Scan to follow and connect. Developer tutorials, real-world use cases, and cutting-edge technologies are continuously updated — with expert support from leading engineers and researchers.



Thanks!

